INDUSTRIAL TRAINING KIT MILESTONE OVERVIEW

CONTENT-BASED PRODUCT RECOMMENDATION SYSTEM

AI RECOMMENDATION LOGIC & PATTERN ALIGNMENT

Project Domain:	Artificial Intelligence & NLP Engineering (Project 3 / Batch 2026)
Prepared By:	**MUHAMMAD ZEESHAN**
Industry Framework:	DecodeLabs Industrial Training Standards & Quality Guidelines
Document Date:	June 2026

1. Executive Summary & Core Project Details

1.1 Project Objective

In modern e-commerce structures, customers face severe cognitive friction due to information overload across expansive catalogs. This project implements a fully operational, text-driven **Content-Based Product Recommendation System** using the comprehensive Amazon Fine Food Reviews dataset.

The absolute objective of this architecture is to extract rich descriptive features ("tags") from user summaries and detailed review bodies, mapping unstructured text sequences directly into multi-dimensional vector spaces. By computing angular distances across candidate vectors, the engine outputs highly accurate recommendations in real time. This explicit metadata matching entirely bypasses the traditional user tracking **"cold-start" limitations**, surfacing items immediately upon catalog insertion based purely on contextual features.

1.2 Tools & Technologies Used

The underlying systems architecture was developed using a production-ready, highly optimized Python data engineering ecosystem:

- **Programming Core:** Python 3.x (Core execution, structural loops, and function matrix definitions)
- **Data Structures & Storage:** `pandas` (High-speed relational dataframe modeling, category casting, data persistence filters) & `numpy` (Multi-dimensional numerical vector handling)
- **Natural Language Processing (NLP):** `re` (Regex string filtration loops) & `sklearn.feature_extraction.text.TfidfVectorizer` (Linguistic normalization, vocabulary mapping, global inverse document frequency weighting)
- **Mathematical Vector Distance:** `sklearn.metrics.pairwise.cosine_similarity` (Directional scale-invariant distance measurements)
- **Memory Efficiency Layers:** `scipy.sparse.coo_matrix` (Compressed coordinate mapping for space-saving storage arrays)
- **Analytical Charts Generator:** `matplotlib.pyplot` & `seaborn` (Side-by-side performance data visualizations)
- **Data Procurement Delivery:** `KaggleApi` (Direct extended endpoint authentication and programmatic streaming of zipped assets)

1.3 Steps Performed

The end-to-end algorithmic implementation follows five sequential phases to process raw content strings into clean, sorted recommendations:

1. **Dataset Acquisition & Integrity Validation:** Programmatically connects to the Kaggle dataset cluster, pulls the underlying compressed archive, and unpacks the target CSV structure. The code handles missing textual attributes inside metadata blocks by overwriting null values with static string literals ("Unknown") and runs duplicate validation logs.
2. **Schema Refactoring & Factorization:** Re-indexes column mappings into uniform naming conventions (`ProductId` to `ProdID`, `Score` to `Rating`). Executes memory-saving categorical factorization via `pd.factorize()` to map complex alphanumeric identifiers to optimized numerical pointer tables.
3. **NLP Filtering & Tag Synthesis:** Merges review titles and bodies. Employs a text-cleansing regex regular mask `([a-z]+)` to force lowercase matching and eliminate formatting artifacts, digits, and punctuation. Drops generic syntax tokens via `ENGLISH_STOP_WORDS` and slices text arrays to pull the top 30 key descriptors into unified product tokens.
4. **TF-IDF Coordinate Mapping:** Feeds the custom tags into a fitted `TfidfVectorizer` configuration, yielding sparse weight vectors. This mathematical layer scales token significance, reducing the influence of broad, generic words while highlighting distinct product descriptive identifiers.
5. **Vector Angle Proximity Retrieval:** Captures interactive keyword queries from the console layer, matches partial string configurations, extracts the item's coordinate vector, and executes dot-product similarity rankings against the remaining dataset rows to return the top 8 closest recommendations.

2. Natural Language Processing & Vector Space Geometry

The underlying mathematical modeling transforms natural human language records into high-dimensional coordinate values. This quantitative tracking replaces arbitrary keyword rule-matching with a deterministic spatial distance framework.

2.1 Text Synthesis & Normalization Math

For each individual row tracking index i inside the collection, a combined text field is assembled by compounding distinct textual strings into a singular, cohesive corpus representation:

$$\text{Linguistic Field Composition: } C_i = Summary_i + " " + Text_i$$

The regular expression script isolates specific alphabet strings while stripping numbers and punctuation marks. It applies an explicit vocabulary-matching conditional step to cross-reference every token against the standard `ENGLISH_STOP_WORDS` dictionary. This prunes high-frequency, non-descriptive grammar (e.g., pronouns, articles, conjunctions) to leave a high-density tag representation containing up to 30 unique words.

2.2 Term Frequency-Inverse Document Frequency Weighting

To prevent global generic terms from skewing matching results, text coordinates are calculated using Term Frequency-Inverse Document Frequency (TF-IDF) transformations. This mathematically down-weights terms that appear frequently across the entire dataset while accentuating specific local descriptive variables.

The weight for a targeted term t inside a local document tag set d across the total corpus landscape D is computed as follows:

$$TF\text{-}IDF(t, d, D) = TF(t, d) \times IDF(t, D)$$

Where Term Frequency (TF) tracks the absolute occurrence rate of that keyword token within the item's local tags, and Inverse Document Frequency (IDF) computes its distinctiveness across the entire global catalog:

$$IDF(t, D) = \log\left[(1 + |D|) / (1 + |\{d \in D : t \in d\}|) \right] + 1$$

2.3 Cosine Similarity Matrix Evaluation

The angular alignment between a selected item feature vector A and any arbitrary candidate item vector B from the database is evaluated by calculating the cosine of the angle separating their trajectories. This scale-invariant measurement depends entirely on tag composition rather than review lengths, preventing verbose feedback from artificially bloating similarity results:

$$\text{Cosine Proximity}(\theta) = (A \cdot B) / (||A||\,||B||) = (\sum A_i B_i) / (\sqrt{\sum A_i^2} \times \sqrt{\sum B_i^2})$$

This formula maps scores to a strict, bounded decimal space between 0.0000 (complete perpendicular discrepancy) and 1.0000 (absolute spatial vector match alignment), establishing an objective mathematical rank for catalog recommendations.

MATHEMATICAL LAYER	CORE ALGORITHMIC IMPLEMENTATION	SYSTEM OPTIMIZATION IMPACT
Linguistic Tokens Filter	Regex filtering + `ENGLISH_STOP_WORDS` pruning	Eliminates textual noise and limits feature dimensions.
TF-IDF Vectorizer	Sparse coordinate tracking via `coo_matrix` structures	Balances global term frequencies to highlight unique product identifiers.
Cosine Angular Metric	Dot-product arrays calculated via `cosine_similarity()`	Delivers fast, scale-invariant text matching in real time.

3. Results & Output Obtained

The system pipeline integrates textual analysis, mathematical modeling, and visualization components into a clean, interactive workflow. When an end-user provides a search query or a partial keyword entry, the application identifies matching products, computes similarity metrics, and ranks the recommendations.

3.1 Runtime System Execution Logs

When executed, the console block creates an interactive environment. It accepts partial keyword strings, cross-references internal database matrices, computes angular similarity arrays, and prints matching items instantaneously. The following data capture represents the exact structural text-based results printed by the execution runtime engine:

```
Enter a product name or keyword to get recommendations: Instant Oatmeal
Matched: 'Instant Oatmeal Variety Pack'
Top recommendations for: 'Instant Oatmeal'

        ProductName                                        Rating   Similarity
28133   Typical instant oatmeal - typical instant oatm...      3       0.5911
64951   Typical instant oatmeal - typical instant oatm...      3       0.5911
348588  Best instant oatmeal out there - best instant ...      5       0.5819
151843  McCann's Irish Oatmeal in instant packets - mc...      5       0.5773
34      Great Irish oatmeal for those in a hurry! - gr...      5       0.5695
346926  Glutenfreeda's Instant Oatmeal, Natural - glut...      5       0.5631
444991  Instant Oatmeal - instant oatmeal simply best ...      5       0.5588
151884  The World's Best Instant Oatmeal - world s bes...      5       0.5584
```

3.2 Visual Subplot Analytics Interpretation

To present recommendation outputs clearly to commercial stakeholders, the script compiles tabular metrics and visualizes them using a dual-panel horizontal bar chart via `plt.subplots(1, 2)`. This layout separates consumer approval metrics from pure mathematical similarity scores.

3.3 Recommendation Performance Visualization Map

To fulfill project requirements, performance indices are integrated directly into the evaluation section using the uploaded content chart visualization asset:

FIGURE 3.1: PROJECT 3 PRODUCT RECOMMENDATION ANALYTICS MAP

	ProductName	Rating	Similarity
28133	Typical instant oatmeal - typical instant oatm...	3	0.5911
64951	Typical instant oatmeal - typical instant oatm...	3	0.5911
348588	Best instant oatmeal out there - best instant ...	5	0.5819
151843	McCann's Irish Oatmeal in instant packets - mc...	5	0.5773
34	Great Irish oatmeal for those in a hurry! - gr...	5	0.5695
346926	Glutenfreeda's Instant Oatmeal, Natural - glut...	5	0.5631
444991	Instant Oatmeal - instant oatmeal simply best ...	5	0.5588
151884	The World's Best Instant Oatmeal - world s bes...	5	0.5584

Execution output visualization displaying the side-by-side subplot tracking metrics for Recommended Products.

4. Project Conclusion & Market Viability

The finalized Artificial Intelligence content filtering system delivers an optimized engine that builds structured mathematical representations from unstructured customer logs. By linking automated regular expression cleansing loops, dynamic inverse frequency word weighting, and high-speed multi-dimensional cosine dot products, the system provides accurate suggestions within milliseconds.

Because this text-driven engine operates independently of historical transactional data or user transaction tracking matrices, it offers an immediate, market-ready framework for digital catalogs, corporate inventory platforms, and e-commerce ecosystems seeking to eliminate catalog friction and maximize user discovery.